

ER Patient Management System Report

Date: January 12, 2026

Course: Programming with Linear Structures

University: Universidad Europea de Madrid

Group members:

- Wesley Lim
- Waseem Adawin
- Christopher Clarke

1. Introduction

Hospital emergency rooms must treat patients according to medical urgency while still respecting the order in which patients arrive. This project implements a complete Emergency Room Patient Management System using a dynamically implemented priority queue and a GTK-based graphical user interface (GUI).

The system allows hospital staff to admit patients, view the next patient to be treated, dequeue patients, print the full queue, and check whether the queue is empty. All operations are performed through an intuitive GUI and backed by a robust linked-list priority queue.

This report explains the problem, the data structures used, the algorithms implemented, the GUI design, additional helper functions, and the results obtained during testing.

2. Problem statement

The goal is to design and implement a patient management system for an emergency room using a priority queue implemented with a linked list.

Each patient has:

- A priority level from 1 (highest urgency) to 5 (lowest urgency)
- A name
- A reason for visit
- An arrival order counter to enforce FIFO behavior among equal-priority patients

The system must support:

- Enqueue (admit patient)
- Dequeue (care next patient)
- Peek (announce next patient)
- Print queue
- Check if empty
- Memory cleanup on exit

A graphical interface must allow users to perform these operations interactively.

3. Data structures

Patient

Stores:

- name (string)
- reason (string)
- priority (1–5)
- arrival_order (integer counter)

The arrival order ensures FIFO behavior for patients with equal priority.

Node

A linked-list node containing:

- A Patient struct
- A pointer to the next node

PriorityQueue

Contains:

- A pointer to the **front** of the list
- A global counter incremented on each admission

This structure supports dynamic growth and efficient insertion based on priority.

4. Algorithms

Enqueue

Inserts a new patient into the queue while maintaining:

- **Priority ordering** (1 is highest urgency)
- **FIFO ordering** for equal priorities

Algorithm steps:

1. Create a new node.
2. If the queue is empty, insert at the front.
3. Otherwise, traverse the list:
 - Stop before the first node with **greater priority number**.
 - For equal priority, skip all nodes with the same priority to preserve FIFO.
4. Insert the new node at the correct position.

Dequeue

Removes and returns the patient at the **front** of the queue.

Used for “Care next patient”.

Peek

Returns the front patient **without removing** them.

Used for “Announce next patient”.

Print

Traverses the queue from front to back and formats each patient into a readable string for the GUI log.

Is Empty

Returns whether the queue contains any patients.

Free Queue

Traverses the list and frees all dynamically allocated nodes.

5. Additional Functions

The assignment allowed extra functions if justified. The following helper functions were added to improve safety, modularity, and GUI integration.

safe_enqueue()

Validates:

- Priority is between 1 and 5
- Name and reason are not empty
- Memory allocation succeeded

Prevents invalid data from reaching the queue.

- **Justification:** Required for defensive programming and GUI safety.

queue_to_string()

Builds a formatted string representation of the entire queue.

- **Justification:** The GUI needs a string to display in the text log.

print_queue()

Prints queue contents to a FILE* stream.

- **Justification:** Keeps printing logic modular and reusable.

validation.c helpers

Centralized input validation functions.

- **Justification:** Avoids duplicated logic and improves maintainability.

output.c helpers

Formats patient information consistently.

- **Justification:** Ensures uniform output across GUI operations.

GUI callback functions

Each menu item has a corresponding callback.

- **Justification:** Required by GTK's event-driven architecture.

6. GUI (GTK) Design and Usage

The GUI is implemented entirely in src/main.c using **GTK3**.

Its design focuses on clarity and ease of use for hospital staff.

Menu Bar

Patients — Admit patient

- Opens a dialog to enter Name, Reason, and Priority.

Patients — Care next patient

- Performs a dequeue and logs the result.

Patients — Announce next patient

- Performs a peek.

Queue — Print full queue

- Displays the entire queue in the log.

Queue — Check if empty

- Reports whether the queue contains patients.

File — Quit

- Frees memory and closes the application.

Scrollable Log

A `GtkTextView` displays:

- Admissions
- Dequeues
- Announcements
- Queue prints
- System messages

Admission Dialog

Collects:

- Name
- Reason
- Priority (1–5)

Input is validated before enqueueing.

7. Computational Implementation

Memory Management

All nodes are dynamically allocated using `malloc`.

On exit, the program frees:

- Every node in the queue
- All GTK widgets
- All temporary buffers

Data Flow

1. User selects an action in the GUI.
2. GTK triggers a callback function.
3. The callback calls the appropriate queue function.
4. The result is formatted and appended to the GUI log.

FIFO Tie-Breaking

Each patient receives an incrementing `arrival_order`.

This ensures:

- Priority determines urgency
- Arrival order determines fairness among equal priorities

Integration

The GUI interacts directly with the backend queue functions.

All logic is separated cleanly:

- Backend handles data and algorithms
- GUI handles user interaction and display

8. Results and Discussion

The system was tested extensively through the GUI.

Test Scenarios

- **Mixed priorities**
 - Higher-urgency patients always appear first.
- **Equal priority FIFO**
 - Patients with the same priority are treated in arrival order.
- **Dequeue correctness**
 - The correct patient is always removed.
- **Peek correctness**
 - The next patient is shown without modifying the queue.
- **Empty queue behavior**
 - The system prevents invalid operations and displays appropriate messages.
- **Queue printing**
 - The full queue is displayed clearly in the log.
- **Memory cleanup**
 - All nodes are freed on exit.

Observations

- The system behaves exactly as required by the assignment.
- The GUI makes the queue operations intuitive and easy to verify.
- The additional helper functions significantly improve reliability and maintainability.

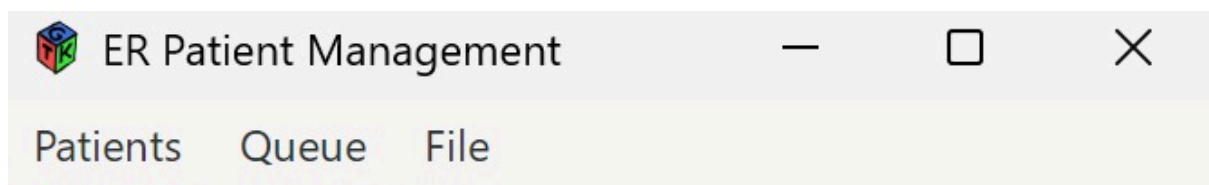
10. Conclusion

This project successfully implements a fully functional emergency room patient management system using a dynamic priority queue and a GTK-based GUI. The system meets all assignment requirements, handles edge cases correctly, and provides a clean, user-friendly interface for interacting with the queue.

The modular design, additional helper functions, and careful memory management ensure that the system is robust, maintainable, and easy to extend in the future.

Some screenshots of the system

Main GUI



SYSTEM: Session started

Patients > Admit Patient

ER Patient Management

Patients Queue File

SYSTEM: Session started

Admit patient

Name

Reason

Priority (1-5)

1

-

+

Cancel

Admit

ER Patient Management

Patients Queue File

SYSTEM: Session started

Admit patient

Name

Reason

Priority (1-5)

3

-

+

Cancel

Admit

Added to GUI:

 ER Patient Management

Patients Queue File

SYSTEM: Session started
ADMIT: John | Priority 3 | Reason: Severe headache

A few more examples

 ER Patient Management

Patients Queue File

SYSTEM: Session started
ADMIT: John | Priority 3 | Reason: Severe headache
ADMIT: Jane | Priority 1 | Reason: Chest pain
ADMIT: Juan | Priority 4 | Reason: Sprained ankle
ADMIT: Carlos | Priority 2 | Reason: High fever
ADMIT: Sara | Priority 5 | Reason: Minor cut
ADMIT: Olivia | Priority 1 | Reason: Asthma

Queue > Print full queue (it shows a queue snapshot)

 ER Patient Management

Patients Queue File

SYSTEM: Session started
ADMIT: John | Priority 3 | Reason: Severe headache
ADMIT: Jane | Priority 1 | Reason: Chest pain
ADMIT: Juan | Priority 4 | Reason: Sprained ankle
ADMIT: Carlos | Priority 2 | Reason: High fever
ADMIT: Sara | Priority 5 | Reason: Minor cut
ADMIT: Olivia | Priority 1 | Reason: Asthma

Print full queue

Check if empty

...Shows queue snapshot

QUEUE SNAPSHOT:
1) Jane | P1 | Chest pain
2) Olivia | P1 | Asthma
3) Carlos | P2 | High fever
4) John | P3 | Severe headache
5) Juan | P4 | Sprained ankle
6) Sara | P5 | Minor cut

Patients > Announce next patient

NEXT: Jane | Priority 1 | Reason: Chest pain

Patients > Care next patient (The patient is dequeued)

NEXT: Jane | Priority 1 | Reason: Chest pain

DEQUEUE: Jane | Priority 1 | Reason: Chest pain

Queue > Print full queue (Here you can see the Queue updated successfully)

QUEUE SNAPSHOT:

- 1) Olivia | P1 | Asthma
- 2) Carlos | P2 | High fever
- 3) John | P3 | Severe headache
- 4) Juan | P4 | Sprained ankle
- 5) Sara | P5 | Minor cut

Queue > Check if queue empty

STATUS: Queue has patients

Showcase when you care for all patients (dequeue all of them) and check if the queue is empty again:

QUEUE SNAPSHOT:

- 1) Olivia | P1 | Asthma
- 2) Carlos | P2 | High fever
- 3) John | P3 | Severe headache
- 4) Juan | P4 | Sprained ankle
- 5) Sara | P5 | Minor cut

STATUS: Queue has patients

DEQUEUE: Olivia | Priority 1 | Reason: Asthma

DEQUEUE: Carlos | Priority 2 | Reason: High fever

DEQUEUE: John | Priority 3 | Reason: Severe headache

DEQUEUE: Juan | Priority 4 | Reason: Sprained ankle

DEQUEUE: Sara | Priority 5 | Reason: Minor cut

DEQUEUE: Queue empty

QUEUE SNAPSHOT:

(empty)

STATUS: Queue is empty

Final Result:



ER Patient Management

Patients Queue File

SYSTEM: Session started

ADMIT: John | Priority 3 | Reason: Severe headache

ADMIT: Jane | Priority 1 | Reason: Chest pain

ADMIT: Juan | Priority 4 | Reason: Sprained ankle

ADMIT: Carlos | Priority 2 | Reason: High fever

ADMIT: Sara | Priority 5 | Reason: Minor cut

ADMIT: Olivia | Priority 1 | Reason: Asthma

QUEUE SNAPSHOT:

1) Jane | P1 | Chest pain

2) Olivia | P1 | Asthma

3) Carlos | P2 | High fever

4) John | P3 | Severe headache

5) Juan | P4 | Sprained ankle

6) Sara | P5 | Minor cut

NEXT: Jane | Priority 1 | Reason: Chest pain

DEQUEUE: Jane | Priority 1 | Reason: Chest pain

QUEUE SNAPSHOT:

1) Olivia | P1 | Asthma

2) Carlos | P2 | High fever

3) John | P3 | Severe headache

4) Juan | P4 | Sprained ankle

5) Sara | P5 | Minor cut

STATUS: Queue has patients

DEQUEUE: Olivia | Priority 1 | Reason: Asthma

DEQUEUE: Carlos | Priority 2 | Reason: High fever

DEQUEUE: John | Priority 3 | Reason: Severe headache

DEQUEUE: Juan | Priority 4 | Reason: Sprained ankle

DEQUEUE: Sara | Priority 5 | Reason: Minor cut

DEQUEUE: Queue empty

QUEUE SNAPSHOT:

(empty)

STATUS: Queue is empty